

Tulenda A.J Makura
HZ40150P
ISE 2202: Essay Assignment 1

Designing a Cloud-Based E-learning Platform

Section A: Architecture Design

The rapid growth of digital education has necessitated the development of scalable, reliable and high performance e-learning platforms capable of supporting thousands of concurrent users. Such systems must handle diverse functionalities including user authentication, content delivery, assessments and real-time interactions. Therefore for this cloud-based e-learning system the most suitable software architecture style is microservices. This approach structures the application as a collection of loosely coupled and independently deployable services. (Newman 2021).

Justification

Scalability, the platform must handle sudden surges in ~~concurrent~~ concurrent users, such as during live lectures or exam periods. A microservice architecture allows for fine-grained ~~set~~ scaling. For instance the video streaming service would experience high load during lectures and can be scaled independently without provisioning additional resources for services like user management or forum service. This is a fundamental advantage over monolithic architectures where the entire application must be scaled leading to inefficient resource utilisation. (Richardson 2018).

Maintainability, the system will be complex and require continuous evolution adding new features like gamification or ~~integration~~ integrating new payment gateways. Microservices provide

maintainability by enforcing bounded contexts. A development team can work on the assessment service without impacting or ~~under~~ understanding the intricacies of the notification service. This decoupling reduces the cognitive load on developers, enables smaller, more focused teams and accelerates the development lifecycle through independent deployment pipelines (Fowler and Lewis 2014).

Performance, while microservices introduce network latency, they offer significant opportunities for performance optimisation. Services can be implemented using the most performant technology stack for their specific task. Furthermore, caching strategies can be implemented per service and services can be ~~geogtra~~ geographically distributed via a ~~content~~ Content Delivery Network (CDN) to bring data closer to users thereby reducing latency and improving overall system responsiveness (Bass et al. 2021).

Section B: Architectural Views

To communicate the system effectively to different stakeholders, the 4+1 architectural view model (Kruchten 1995) is applied using three key views:

Logical view is the view that focuses on the system's functional structure including key classes such as User, Course, Lesson, Enrollment and Assessment. It represents the static organisation of the system and is mainly used by developers and analysts to ensure the design aligns with user requirements and expected functionality.

Process view, this view describes the system's runtime behavior including concurrency and communication between services. It illustrates how microservices interact asynchronously.

through events e.g. an environment event triggering notifications and analytics updates. It is important for understanding performance, scalability and fault tolerance.

Deployment view, this view shows how the system is physically deployed on cloud infrastructure. It includes containerisation (Docker), ~~orke~~ orchestration (Kubernetes), databases, load balancers and CDNs. It helps system administrators understand scalability, system distributions distributions and resilience.

Together these views provide a comprehensive understanding of the system's structure, behaviour and deployment ensuring clarity for developers, engineers and stakeholders.

Section C: Quality Attributes

Quality attributes define the system's non-functional ~~on~~ requirements. The key attributes include:

Scalability, ~~ach~~ achieved through horizontal scaling of microservices using Kubernetes, which adjusts resources based on demand. Measured by request throughput and resources utilisation, aiming for consistent performance as load increases.

Availability is ensured by deploying redundant services across multiple zones using failover mechanisms and reliable messaging systems. Measured by uptime (targeting 99.99%) and Mean Time To Recovery.

Security which is implemented using a defence-in-depth

approach including OAuth 2.0 authentication, data encryption (TLS and AES-256) and Role-Based Access Control (RBAC). Measured by vulnerability resolution rates, incident response time and audit success.

Performance, improved through caching, database optimisation and asynchronous processing. Measured by latency, throughput and ~~defect~~ ~~adherence~~ adherence to

Maintainability, achieved via modular microservices, well-documented APIs, logging and CI/CD pipelines with ~~1~~ automated testing. Measured by development ~~egs~~ cycle time, code changes and defect resolution time.

Section D: Design Patterns

Design patterns provide proven solutions to ~~near~~ recurring design problems (Gamma et al 1995)

1. Pattern: Saga (for distributed transactions)

The problem is that a single business ~~off~~ operation such as a user enrolling in a paid course, spans multiple microservices. Ensuring data consistency across these services without a distributed database transaction is a challenge.

To counter ~~the~~ this, the ~~soa~~ solution is that the saga pattern manages this as a sequence of local transactions. For the enrollment process a choreographed saga is implemented. The Enrollment Service creates a pending enrollment and publishes an event. The payment Service processes the payment

and publishes on a success or failure event. The Enrollment Service then finalises the enrollment and finally the Notification Service sends a confirmation. If the payment fails a compensating transaction is triggered (eg the Enrollment Service deletes the pending enrollment).

2. Pattern: API Gateway

The problem is that a client application (web or mobile) needs to interact with ~~thousands~~ numerous microservices each with its own endpoint. This creates client complexity, exposes the internal service structure and requires handling cross-cutting concerns like authentication and rate limiting in each service.

Solution: An API gateway acts as a single entry point for all clients. It routes requests to the appropriate microservices, aggregates responses and centrally enforces cross-cutting concerns like authentication, SSL termination and rate limiting.

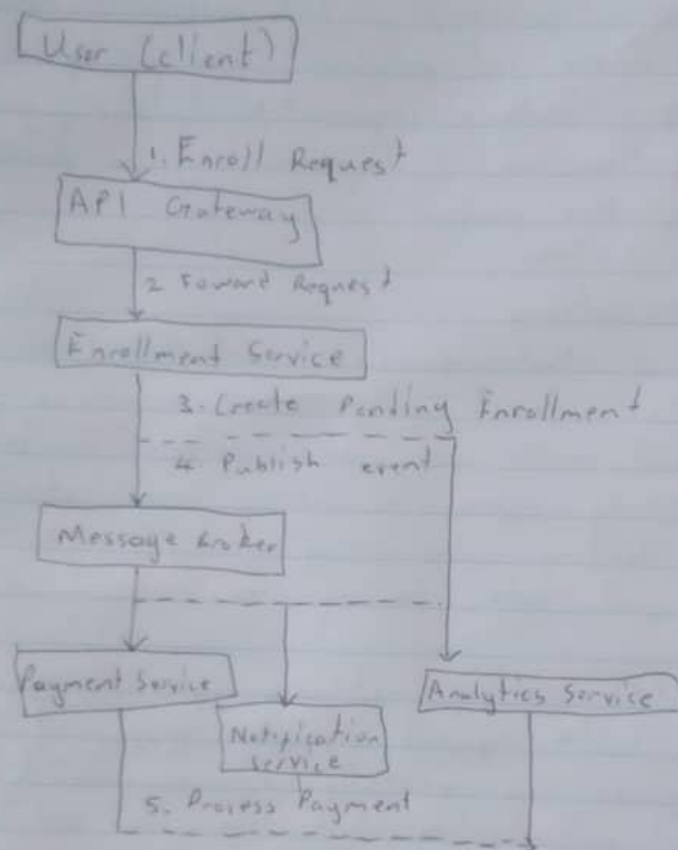
3. PTT Pattern: Circuit Breaker

The problem is that in a distributed system, a ~~microservice~~ microservice might become slow or unresponsive. Without protection clients can waste resources waiting for timeouts leading to cascading failures across the system.

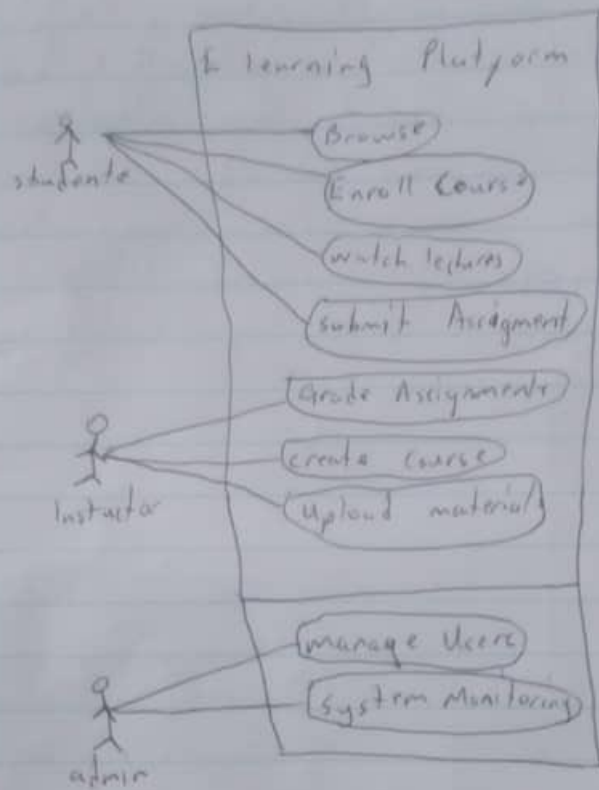
The solution is that the circuit breaker wraps calls to a remote service and monitors for failures. Initially the circuit is closed. When the number of failures exceeds a threshold, the circuit "opens" and

subsequent calls fail immediately without attempting the call. After a timeout the circuit enters a "half-open" state allowing a limited number of test calls to ~~be~~ succeed. If successful, the circuit closes otherwise it remains open. Implementing this in the video streaming ~~server~~ service prevents the user interface from continuously trying to connect to a failing streaming server preserving overall system stability.

Sequence Diagram



Section E : UML



Component Diagram

